Operating Systems — Interview Questions

The OS questions asked in fresher technical interviews — processes, threads, scheduling, deadlock, memory and synchronisation — with short answers to say aloud.

Answer like this in the interview

Define in one line → **explain** with one real-world example → add **one detail** (a difference, a use case or a limitation). Stop at 30–45 seconds and let the interviewer ask the follow-up.

Questions marked ★ **MOST ASKED** come up in almost every fresher technical round — learn those first.

Top questions to learn first

What is an OS · Process vs thread · Process states · CPU scheduling algorithms · Deadlock & its 4 conditions · Banker's algorithm · Paging vs segmentation · Virtual memory · Semaphore vs mutex · Thrashing

1 · Basics

1 What is an operating system? What are its functions? ★ MOST ASKED

System software that acts as an **interface between the user/applications and the hardware** and manages the computer's resources.

Functions: **process management**, **memory management**, **file management**, **device (I/O) management**, **security** and providing a user interface. Examples: Windows, Linux, macOS, Android.

2 What is the kernel? Monolithic vs microkernel?

The **kernel** is the core of the OS that runs in privileged mode and controls CPU, memory and devices. A **monolithic** kernel runs all services in kernel space (Linux) — fast but larger. A **microkernel** keeps only essentials in the kernel and runs other services in user space (QNX, Minix) — more stable and modular.

Types of operating systems

Type	Idea	Example
Batch	Jobs grouped and run without user interaction	Early mainframes
Multiprogramming	Several programs in memory; CPU switches when one waits for I/O	—
Time-sharing / multitasking	CPU time divided into slices among users/programs	Windows, Linux
Real-time (RTOS)	Guaranteed response within a deadline	Airbag controllers, FreeRTOS
Distributed	Many computers appear as one system	—

3 What is a system call?

The way a program **requests a service from the kernel**, switching from user mode to kernel mode — e.g. `fork()`, `exec()`, `read()`, `write()`, `open()`. Categories: process control, file management, device management, information and communication.

2 · Processes & threads

4 What is a process? What is a PCB? ★ MOST ASKED

A **process is a program in execution**. A program is passive code on disk; a process is active, with its own memory (code, data, heap, stack). The OS tracks each process in a **Process Control Block (PCB)**: process ID, state, program counter, CPU registers, memory info, open files and scheduling priority.

5 What are the states of a process? ★ MOST ASKED

New → **Ready** → **Running** → (**Waiting**/Blocked for I/O → back to Ready) → **Terminated**.

Running → Ready happens when the time slice ends or a higher-priority process arrives (pre-emption).

Process vs thread ★ MOST ASKED

Point	Process	Thread
Definition	A program in execution	A lightweight unit of execution inside a process
Memory	Own separate address space	Shares code, data and heap with sibling threads; own stack and registers
Creation / switching	Slower, more expensive	Faster, cheaper
Communication	Needs IPC (pipes, shared memory, messages)	Directly through shared memory
Failure	One crash doesn't affect others	One thread can crash the whole process
Example	Chrome and Word running separately	Tabs/tasks inside one app: UI thread, download thread

6 What is context switching?

Saving the state (PCB) of the running process and loading the state of the next one so the CPU can switch between them. It is pure overhead — no useful work happens during the switch — so too many switches hurt performance.

7 What are zombie and orphan processes?

A **zombie** has finished but its entry stays in the process table because the parent has not read its exit status (`wait()`). An **orphan** is a running process whose parent has terminated; it is adopted by `init/systemd`.

8 Multiprogramming, multitasking, multithreading and multiprocessing?

- **Multiprogramming** — several programs in memory so the CPU is never idle.
- **Multitasking** — CPU time-slices between tasks so they appear to run together.
- **Multithreading** — one process runs several threads.
- **Multiprocessing** — two or more CPUs/cores run processes truly in parallel.

3 · CPU scheduling

9 Pre-emptive vs non-pre-emptive scheduling?

Pre-emptive: the OS can take the CPU away from a running process (time slice over, higher priority arrives) — Round Robin, SRTF. **Non-pre-emptive**: a process keeps the CPU until it finishes or waits for I/O — FCFS, non-pre-emptive SJF.

Scheduling algorithms ★ MOST ASKED

Algorithm	Rule	Pro / con
FCFS	First come, first served (queue)	Simple · **convoy effect** — short jobs wait behind a long one
SJF	Shortest burst time first (non-pre-emptive)	Minimum average waiting time · needs burst time in advance, starvation
SRTF	Pre-emptive SJF — shortest remaining time	Better response · starvation of long jobs
Priority	Highest priority first (either type)	Starvation · fixed by **aging** (raise priority over time)
Round Robin	Each process gets a fixed **time quantum** in turn	Fair, good for time-sharing · too small a quantum → many context switches
Multilevel queue	Separate queues (system, interactive, batch)	Flexible · complex

10 Define the scheduling terms. ★ MOST ASKED

- **Arrival time** — when the process enters the ready queue. **Burst time** — CPU time it needs.
- **Completion time** — when it finishes. **Turnaround time = completion − arrival**.
- **Waiting time = turnaround − burst**. **Response time** = first time on CPU − arrival.

Example (FCFS, all arrive at 0): P1 = 5, P2 = 3, P3 = 2 → waiting 0, 5, 8 → average 13 / 3 ≈ 4.33. SJF order P3, P2, P1 → waiting 0, 2, 5 → average 7 / 3 ≈ 2.33.

11 What is starvation and how is it solved?

A process waits indefinitely because others keep getting the CPU (e.g. a low-priority process in priority scheduling). Solved by **aging** — gradually increasing the priority of waiting processes.

---

4 · Deadlock

12 What is a deadlock? What are the four necessary conditions? ★ MOST ASKED

A situation where processes wait forever, each holding a resource the other needs. Example: P1 holds the printer and wants the scanner; P2 holds the scanner and wants the printer.

All four must hold together (Coffman conditions):

- **Mutual exclusion** — a resource can be used by one process at a time.
- **Hold and wait** — a process holds one resource while waiting for another.
- **No pre-emption** — a resource can't be forcibly taken away.
- **Circular wait** — P1 waits for P2, P2 for P3 … Pn for P1.

---

13 How are deadlocks handled? ★ MOST ASKED

- **Prevention** — break one of the four conditions (e.g. request all resources at once, or number resources and request in order to stop circular wait).
- **Avoidance** — check every request and grant it only if the system stays in a **safe state** (Banker's algorithm).
- **Detection and recovery** — allow deadlock, detect cycles in the resource-allocation graph, then kill a process or pre-empt resources.
- **Ignore it** (ostrich algorithm) — used by most desktop OSs because deadlocks are rare.

---

14 Explain the Banker's algorithm.

A deadlock-**avoidance** algorithm. Each process declares its **maximum** need. Using the Available, Max, Allocation and **Need = Max − Allocation** matrices, the OS grants a request only if there is still a **safe sequence** in which every process can finish. If no safe sequence exists, the request waits. Like a bank that lends only if it can still satisfy every customer.

Deadlock vs starvation

Point	Deadlock	Starvation
What happens	Processes block each other forever	One process waits for a long time while others run
Cause	Circular wait on resources	Unfair scheduling or priority
Fix	Prevention, avoidance, detection	Aging

5 · Memory management

15 What is paging? ★ MOST ASKED

Memory management where **physical memory is split into fixed-size frames** and a **process into pages of the same size**. Pages can sit in any free frame, so memory need not be contiguous. A **page table** maps page numbers to frame numbers. It removes external fragmentation but can cause small internal fragmentation in the last page.

Paging vs segmentation

Point	Paging	Segmentation
Division	Fixed-size pages	Variable-size logical segments (code, stack, data)
Visible to programmer	No	Yes
Fragmentation	Internal	External
Table	Page table	Segment table (base + limit)

16 What is virtual memory? What is a page fault? ★ MOST ASKED

Virtual memory lets a process run even if it doesn't fit fully in RAM — only the needed pages are loaded (**demand paging**); the rest stay on disk (swap). Programs can be larger than physical memory.

A **page fault** happens when a process accesses a page not in RAM; the OS loads it from disk, replacing another page if memory is full.

17 What are the page replacement algorithms?

- **FIFO** — replace the oldest page. Can suffer **Belady's anomaly** (more frames → more faults).
- **LRU** — replace the page **least recently used**. Good in practice.
- **Optimal** — replace the page not needed for the longest time in future; best possible but needs future knowledge, used as a benchmark.

Example: reference string 7 0 1 2 0 3 0 4 with 3 frames → FIFO gives 7 faults, LRU gives 6, Optimal gives 6.

18 What is thrashing? ★ MOST ASKED

When a process has too few frames, it page-faults constantly and the system spends more time **swapping pages than executing** — CPU use drops sharply. Fix: give processes enough frames (working-set model), reduce the degree of multiprogramming.

Internal vs external fragmentation

Point	Internal	External
Meaning	Unused space **inside** an allocated block	Free memory is enough in total but **not contiguous**
Occurs in	Fixed partitions, paging	Variable partitions, segmentation
Fix	Smaller blocks / pages	Compaction, paging

19 What is the TLB?

The **Translation Lookaside Buffer** is a small, fast hardware cache of recent page-table entries. A TLB hit avoids an extra memory access to the page table, speeding up address translation.

6 · Synchronisation

20 What is a race condition and a critical section?

A **race condition** occurs when several processes/threads access shared data at the same time and the result depends on the order — e.g. two threads both incrementing `count`. The **critical section** is the code that touches the shared data; only one process should run it at a time. A solution needs mutual exclusion, progress and bounded waiting.

Semaphore vs mutex ★ MOST ASKED

Point	Mutex	Semaphore
What it is	A lock with ownership	An integer counter with wait() / signal() (P / V)
Allows	One thread at a time	Up to N (counting) or 1 (binary semaphore)
Released by	Only the thread that locked it	Any process/thread
Use	Protect a critical section	Control access to a pool of resources, signalling

21 Name the classic synchronisation problems.

Producer–consumer (bounded buffer), **readers–writers** and **dining philosophers**. Be ready to explain one: in producer–consumer, the producer must wait when the buffer is full and the consumer when it is empty — solved with semaphores `empty`, `full` and a `mutex`.

7 · Files, storage & boot

22 What happens when you switch on a computer (booting)?

Power on → **BIOS/UEFI** runs the POST (hardware check) → finds the boot device → loads the **bootloader** (e.g. GRUB, Windows Boot Manager) → bootloader loads the **kernel** into memory → kernel starts the first process (`init`/`systemd`) and services → login screen.

23 What are disk scheduling algorithms?

They decide the order of disk read/write requests to reduce head movement: **FCFS**, **SSTF** (shortest seek time first), **SCAN** (elevator), **C-SCAN**, **LOOK**. SCAN avoids the starvation that SSTF can cause.

24 What is the difference between RAM, ROM and cache?

RAM is volatile working memory. **ROM** is non-volatile and holds firmware (BIOS). **Cache** is very fast memory close to the CPU (L1, L2, L3) that stores frequently used data to avoid slower RAM access.

Linux commands interviewers ask

`ls`, `cd`, `pwd`, `mkdir`, `rm`, `cp`, `mv`, `cat`, `grep`, `chmod`, `ps`, `top`, `kill`, `df`, `free`. Know `chmod 755` (owner rwx, group and others r-x) and that `ps` lists processes.

Rapid-fire one-liners

Question	One-line answer
What is a dispatcher?	The module that gives the CPU to the process selected by the scheduler.
Long- vs short-term scheduler?	Long-term picks jobs to load into memory; short-term picks the next process for the CPU.
What is a daemon?	A background process with no terminal, e.g. sshd, cron.
What is IPC?	Inter-process communication: pipes, message queues, shared memory, sockets.
What is spooling?	Queueing jobs for a slow device (printer) on disk so the CPU can continue.
What is a deadlock-safe state?	A state where a safe sequence exists for all processes to finish.
What is Belady's anomaly?	More frames causing more page faults — seen with FIFO.
User mode vs kernel mode?	Kernel mode has full hardware access; user programs run with restricted access.
What is swapping?	Moving a whole process between RAM and disk to free memory.
What is fork()?	A system call that creates a child process as a copy of the parent.

Common mistakes interviewers catch

- Saying threads have separate memory — they share the process's code, data and heap.
- Listing only three deadlock conditions — all four are needed together.
- Calling a binary semaphore exactly the same as a mutex — a mutex has ownership.
- Confusing turnaround time (completion − arrival) with waiting time (turnaround − burst).
- Saying paging causes external fragmentation — it causes internal fragmentation.